

Variational Autoencoders

Auto-Encoding Variational Bayes

Diederik P. Kingma

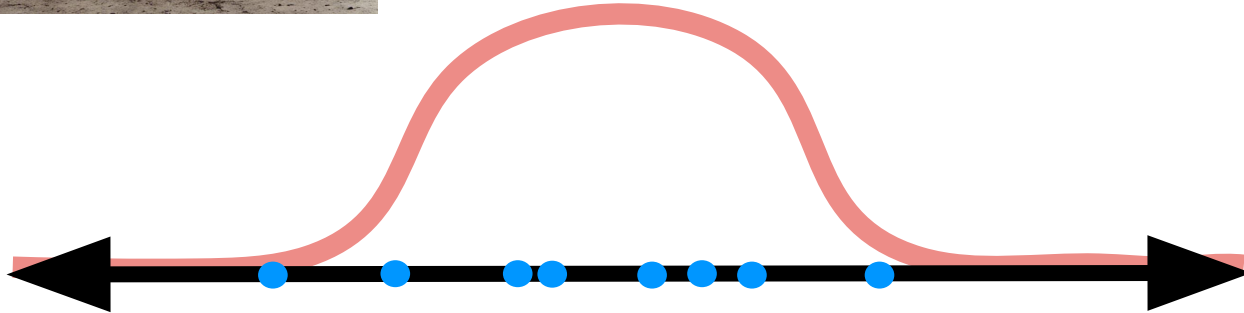
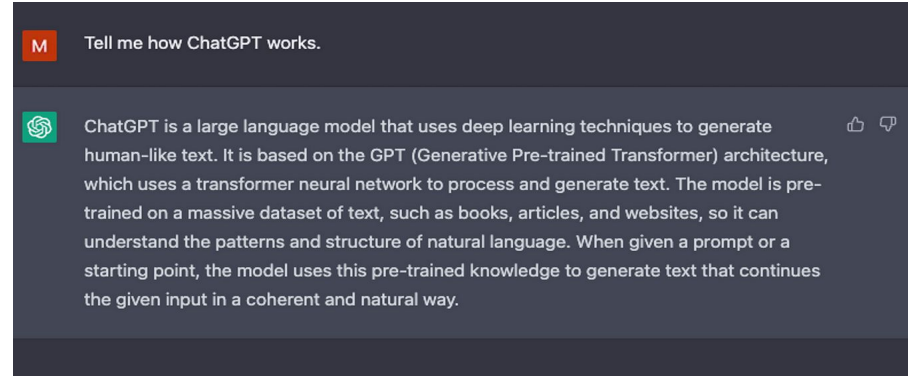
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling

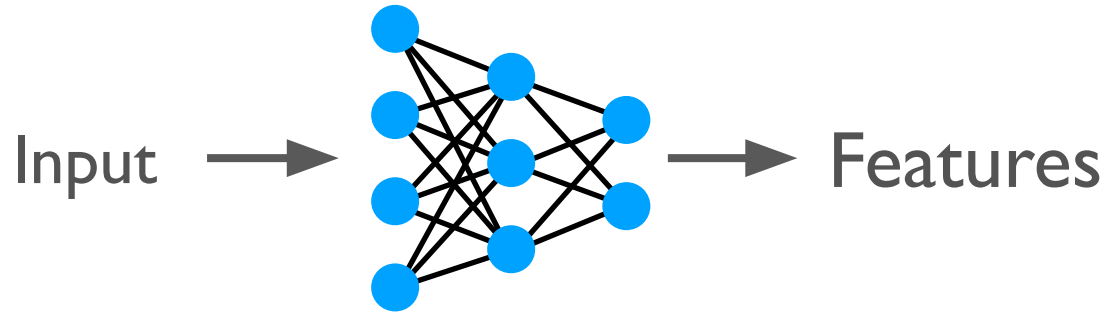
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

GenAI and Modern Deep Learning

Generative Modeling

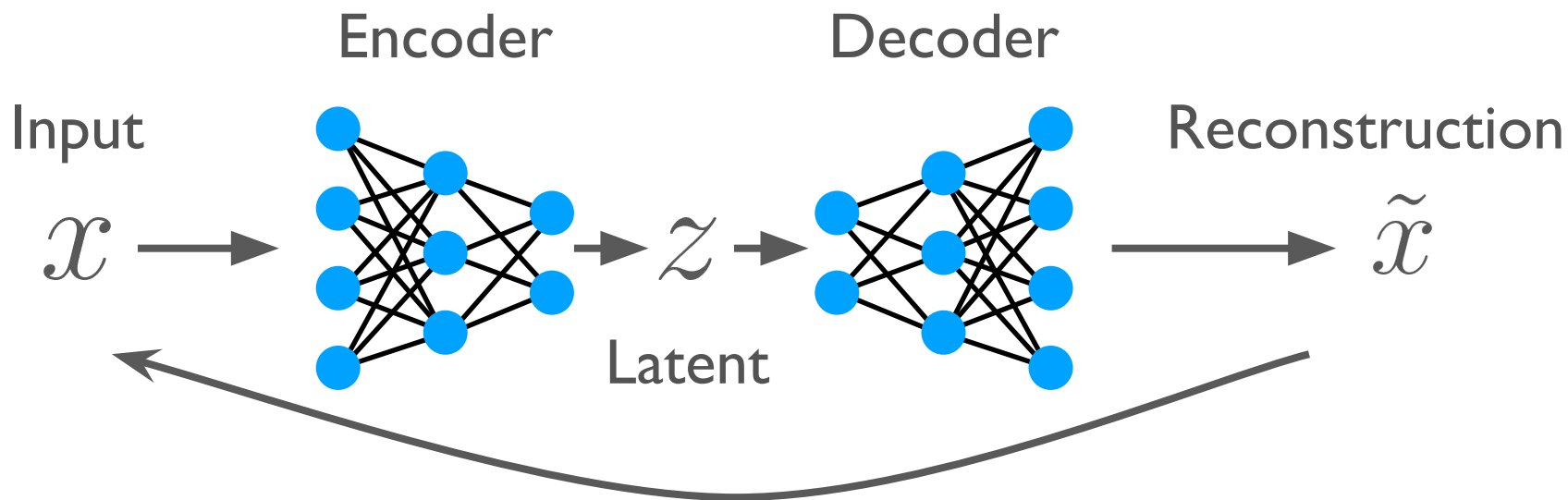


Unsupervised representation learning

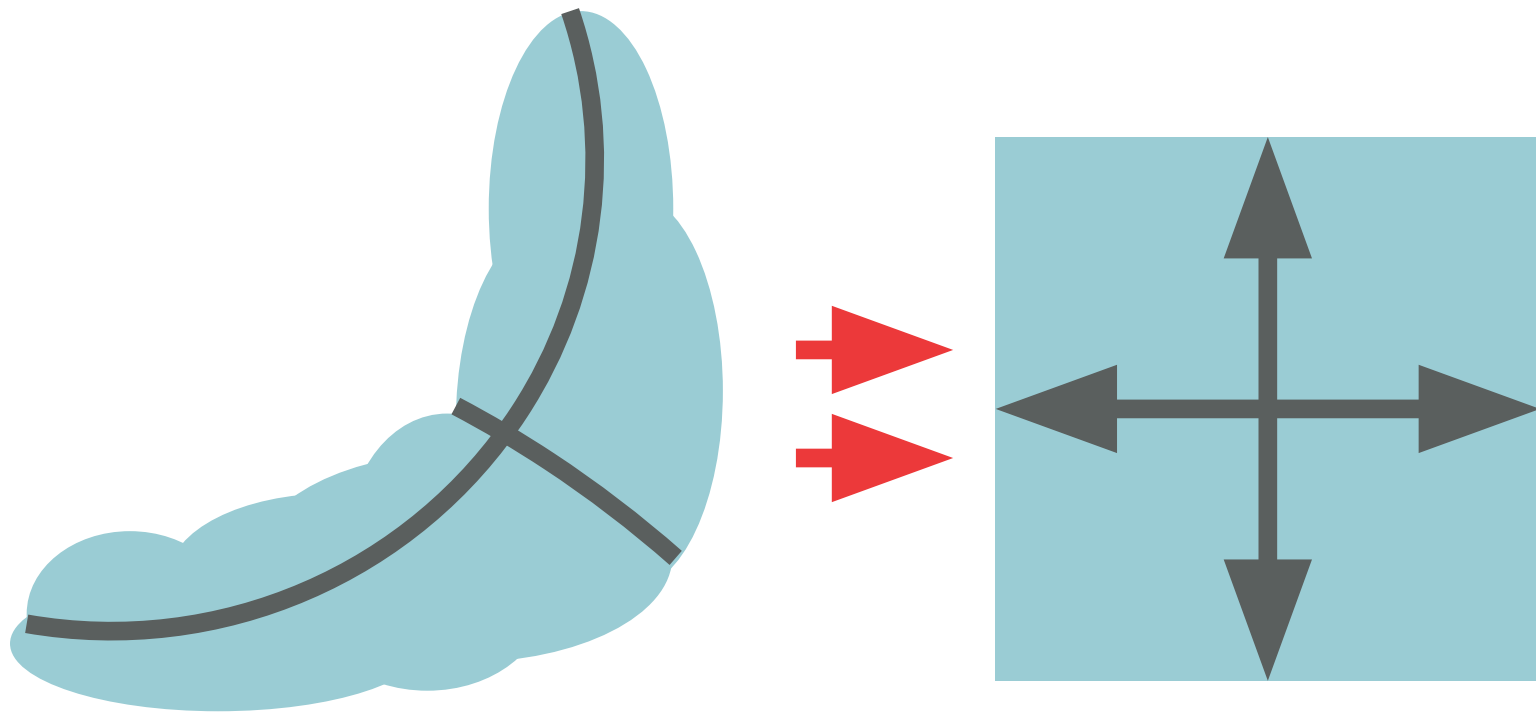


Clustering, compression, classification, detection/segmentation, ...

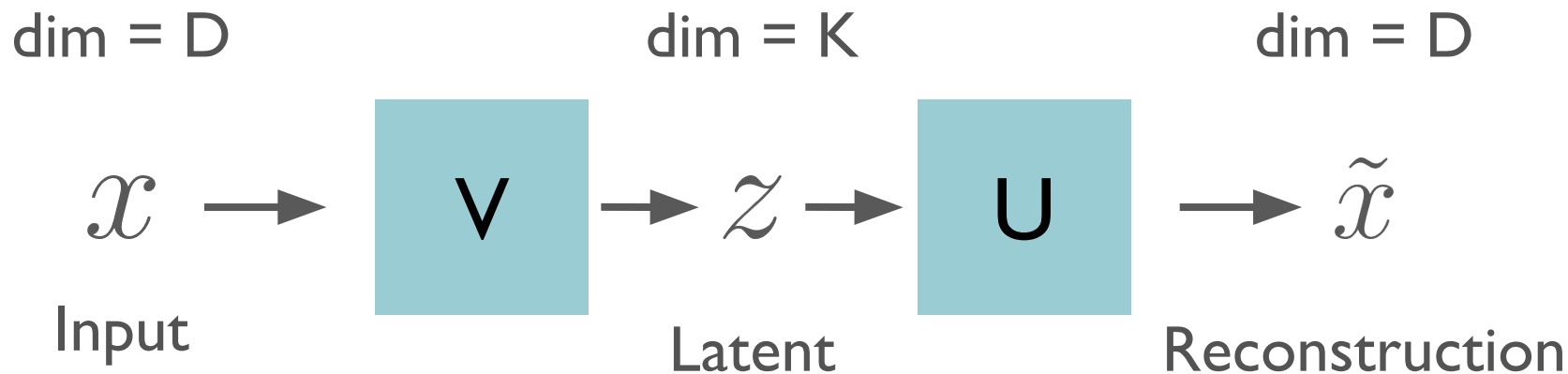
Autoencoders



Disentanglement



A simple linear autoencoder

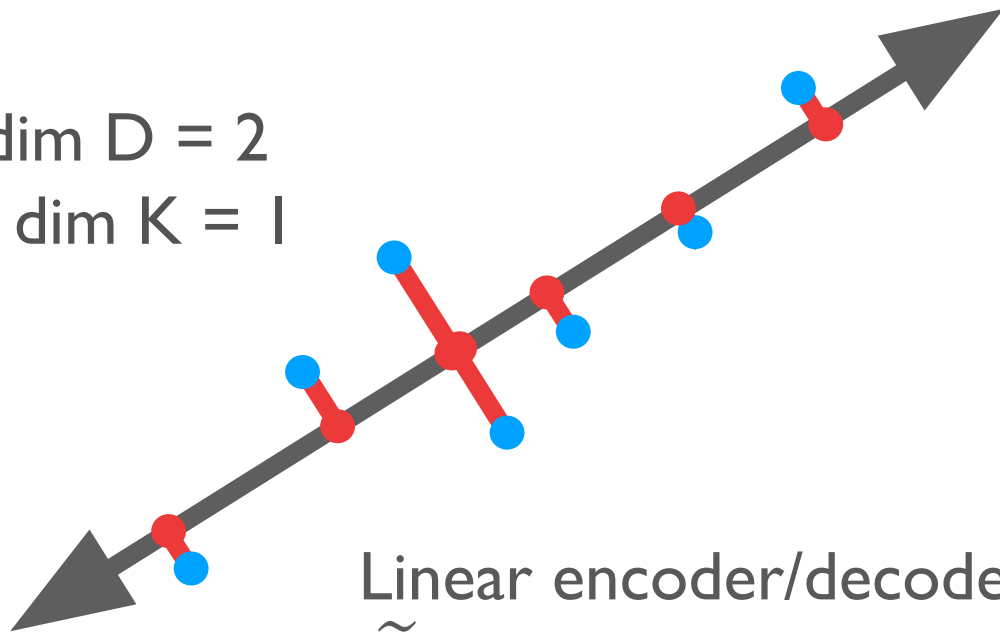


- $\tilde{x} = UVx$
- $\mathcal{L}(U, V) = \mathbb{E}_x[\|x - \tilde{x}\|_2^2]$
- If $K \geq D$, then $UV = I$
- If $K < D$, dimensionality reduction

A simple linear autoencoder

Input dim $D = 2$

Latent dim $K = 1$



Linear encoder/decoder

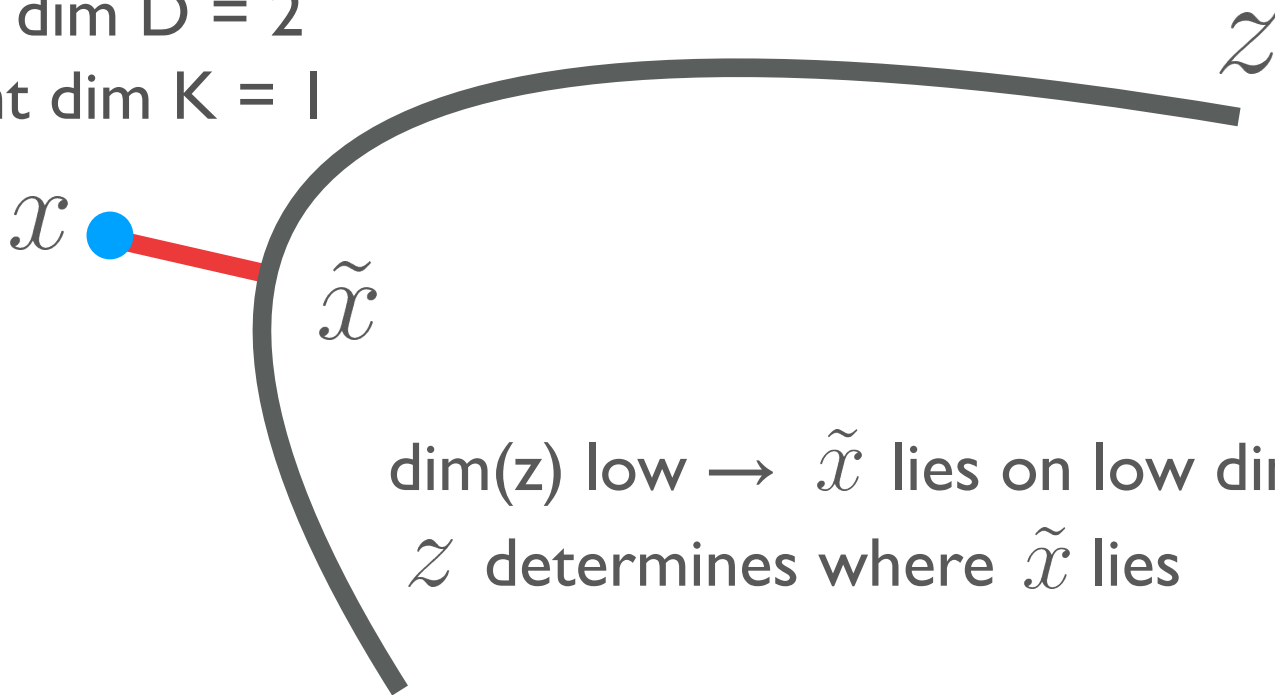
$\tilde{\mathcal{X}}$ lies in low-dimensional linear subspace

$\mathcal{Z}$ parameterizes the subspace

A nonlinear autoencoder

Input dim $D = 2$

Latent dim $K = 1$

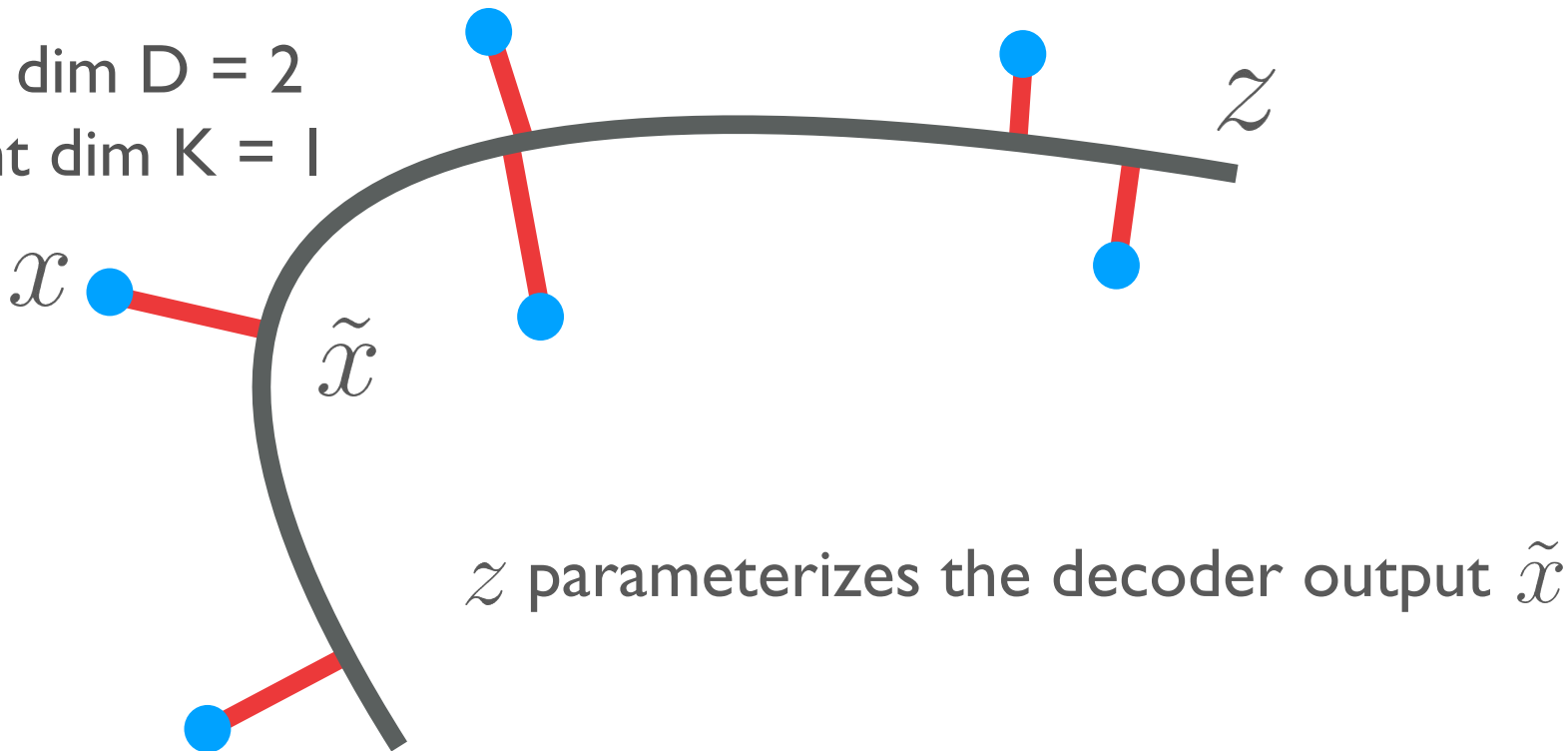


$\dim(z)$ low $\rightarrow \tilde{x}$ lies on low dim manifold
 z determines where $\tilde{x}$ lies

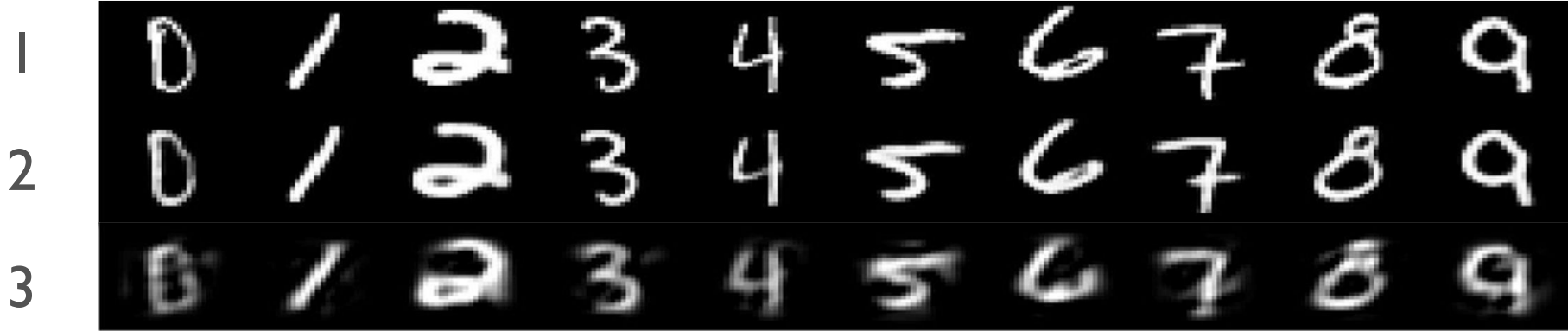
A nonlinear autoencoder

Input dim $D = 2$

Latent dim $K = 1$



Nonlinear autoencoders



1. Data
2. Nonlinear ($K=30$)
3. Linear ($K=30$)

How to generate new samples?

Sample new z and decode

Which z ?



Here's what we'll need:

Architecture

Loss function

Gradients

Train with SGD

Generative model

Decoder: $p_{\theta}(x|z)$

Will be parameterized by a NN $G_{\theta}(z)$

Assume standard normal latent distribution $p(z)$

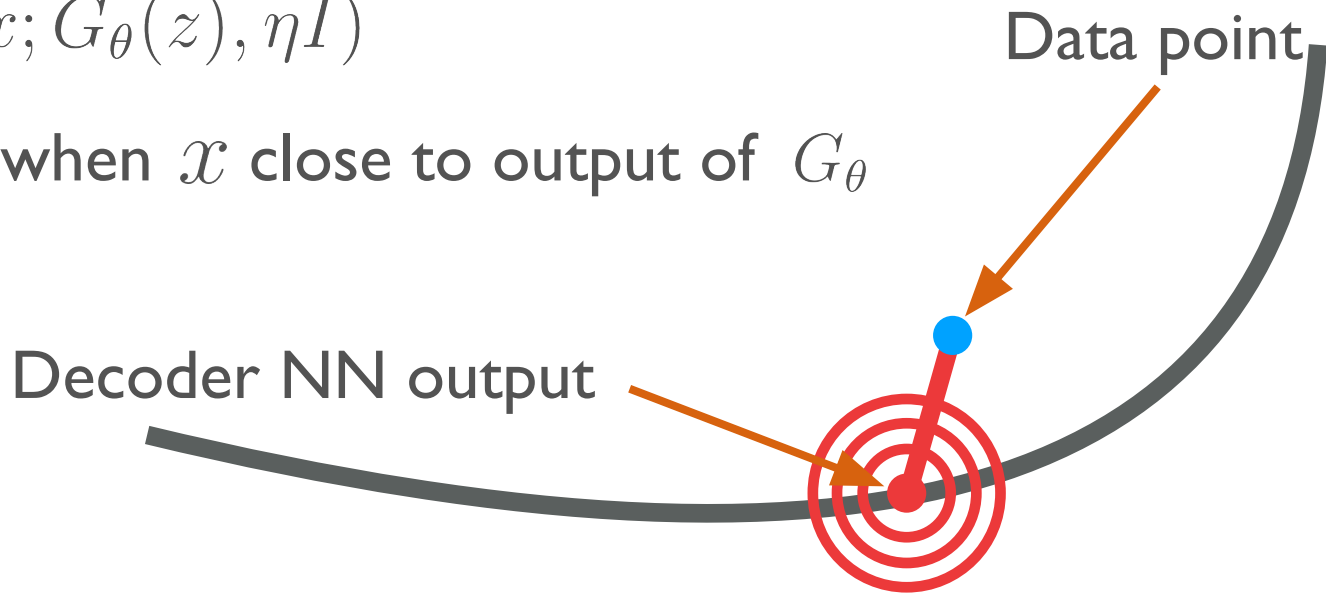
Generative model:
$$p_{\theta}(x) = \int \underbrace{p_{\theta}(x|z)}_{\text{decoder}} \underbrace{p(z)}_{\text{latent}} dz$$

The observation model

Noisy observation model:

$$p_{\theta}(x|z) = \mathcal{N}(x; G_{\theta}(z), \eta I)$$

$p_{\theta}(x|z)$ is high when x close to output of G_{θ}



An intractable integral

Now, we can compute $p_{\theta}(x|z)$

But how to compute:

$$p_{\theta}(x) = \int p_{\theta}(x|z)p(z)dz ?$$

$p_{\theta}(x|z)$ depends on G_{θ} which is complicated 😞

The variational approximation

$$p_{\theta}(x) = \frac{p_{\theta}(x, z)}{p_{\theta}(z|x)} = \frac{\overbrace{p_{\theta}(x|z)}^{\text{decoder}} \overbrace{p_{\theta}(z)}^{\text{latent}}}{p_{\theta}(z|x)}$$

We know the numerator above.

If only we had variational approximation $q_{\phi}(z|x) \approx p_{\theta}(z|x)$

$$\text{Then } p_{\theta}(x) \approx \frac{p_{\theta}(x|z)p_{\theta}(z)}{q_{\phi}(z|x)}$$

$q_{\phi}(z|x)$ will be the encoder

The encoder

Encoder $q_\phi(z|x)$ is a probability distribution

How to make NN output a probability distribution?

Parameterize a Gaussian by output of NN: $F_\phi(x)$

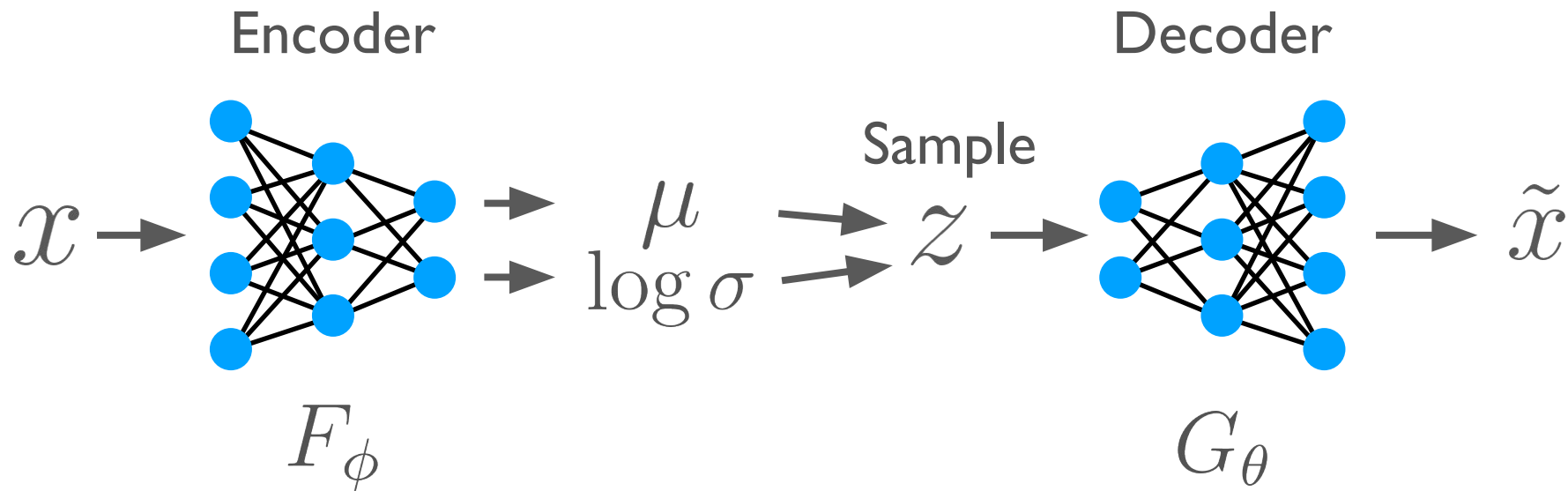
$$F_\phi(x) = \mu_\phi(x), \Sigma_\phi(x)$$

$$q_\phi(z|x) = \mathcal{N}(z; \mu_\phi(x), \Sigma_\phi(x))$$

In practice, let $\Sigma(x) = \text{diag}(\sigma_1(x)^2, \dots, \sigma_n(x)^2)$

Encoder outputs $\log \sigma$

The variational autoencoder



The ELBO

Want to maximize $p_\theta(x)$ but can't compute directly

Maximize a lower bound on $\log p_\theta(x)$ instead

The ELBO

$$\begin{aligned}\log p(x) &= \mathbb{E}_{q(z|x)}[\log p(x)] = \mathbb{E}_{q(z|x)}\left[\log\left[\frac{p(x, z)}{p(z|x)}\right]\right] \\&= \mathbb{E}_{q(z|x)}\left[\log\left[\frac{p(x, z)q(z|x)}{q(z|x)p(z|x)}\right]\right] \\&= \underbrace{\mathbb{E}_{q(z|x)}\left[\log\left[\frac{p(x, z)}{q(z|x)}\right]\right]}_{\text{ELBO}} + \underbrace{\mathbb{E}_{q(z|x)}\left[\log\left[\frac{q(z|x)}{p(z|x)}\right]\right]}_{D_{KL}(q(z|x)||p(z|x))} \\&\geq \mathbb{E}_{q(z|x)}\left[\log\left[\frac{p(x, z)}{q(z|x)}\right]\right]\end{aligned}$$

The ELBO

$$\begin{aligned}\log p(x) &= \underbrace{\mathbb{E}_{q(z|x)} \left[\log \left[\frac{p(x, z)}{q(z|x)} \right] \right]}_{\text{ELBO}} + \underbrace{\mathbb{E}_{q(z|x)} \left[\log \left[\frac{q(z|x)}{p(z|x)} \right] \right]}_{D_{KL}(q(z|x) \| p(z|x)) \geq 0} \\ &\geq \mathbb{E}_{q(z|x)} \left[\log \left[\frac{p(x, z)}{q(z|x)} \right] \right]\end{aligned}$$

The ELBO

$$\text{ELBO} : \mathbb{E}_{q(z|x)} \left[\log \left[\frac{p(x, z)}{q(z|x)} \right] \right] = \log p(x) - D_{KL}(q(z|x) || p(z|x))$$

Maximizing the ELBO:

- (1) maximizes the log-likelihood
- (2) Brings $q(z|x)$ close to $p(z|x)$

The training loss

Let $\mathcal{L}_{\theta,\phi}(x) =$ negative ELBO

$$= D_{KL}(q_{\phi}(z|x) || p_{\theta}(z)) - \underbrace{\mathbb{E}_{q_{\phi}(z|x)} \log p_{\theta}(x|z)}_{\text{NLL}}$$

$$\text{Minimize } \sum_{x \in \mathcal{D}} \mathcal{L}_{\theta,\phi}(x)$$

We will examine each term separately.

The KL-Divergence term

$$\text{Minimize } D_{KL}(q_{\phi}(z|x) || p_{\theta}(z))$$

Take $p_{\theta}(z)$ to be the standard normal distribution

Minimize distance between q_{ϕ} and standard normal

$$\log \sigma \text{ and } \mu \rightarrow 0$$

The NLL term

$$\text{Minimize NLL} \quad -\mathbb{E}_{q_\phi(z|x)} \log p_\theta(x|z)$$

Assume a Gaussian observation model

$$\begin{aligned} -\log p_\theta(x|z) &= -\log \mathcal{N}(x; G_\theta(z), \eta I) \\ &= -\log \left[\frac{1}{(2\pi\eta)^{D/2}} \exp\left(-\frac{1}{2\eta} \|x - G_\theta(z)\|_2^2\right) \right] \\ &= \cancel{\frac{1}{2\eta}} \|x - G_\theta(z)\|_2^2 + \cancel{\text{constant}} \end{aligned}$$

Make output close to $\mathcal{X}$, **Reconstruction Loss**

The VAE loss

To minimize $D_{KL}(q_\phi(z|x) || p_\theta(z))$,

Make $q_\phi(z|x)$ close to $\mathcal{N}(0, I)$

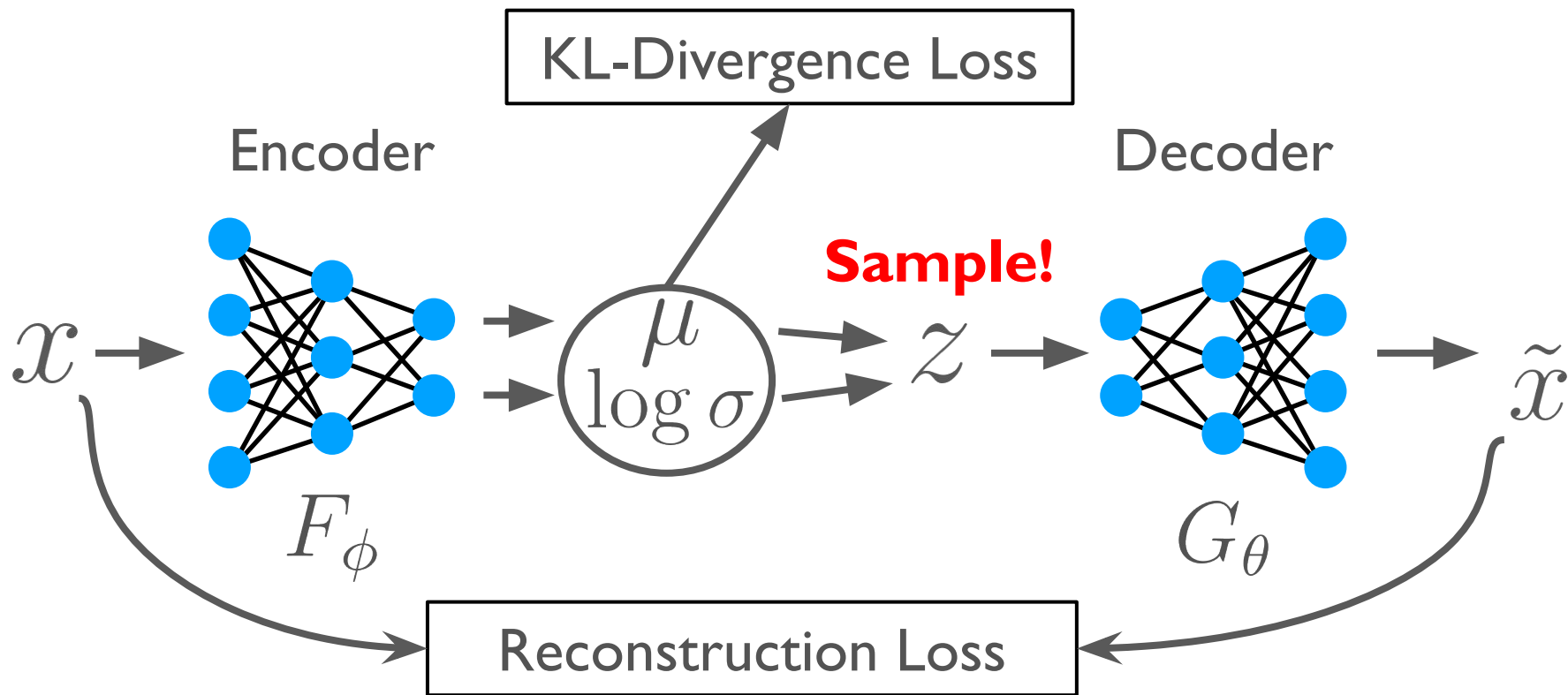
To minimize reconstruction loss $\mathbb{E}_{q_\phi(z|x)} \|x - G_\theta(z)\|_2^2$,
make $q_\phi(z|x)$ a point mass on $\mathcal{Z}$ that minimizes distance

These two terms fight each other.

It's time to train!

Want to train using SGD

How do we compute $\nabla_{\theta, \phi} \mathcal{L}_{\theta, \phi}(x)$?



How to compute gradient through sampling?

The reparameterization trick

Problem: how do we compute $\nabla_{\phi} \mathbb{E}_{q_{\phi}(z|x)} \|x - G_{\theta}(z)\|_2^2$?

Random variable ϵ not depending on ϕ ,

Transform ϵ into z :

$z = h(\epsilon, \phi, x)$, where h is differentiable ϕ

w.r.t.
Sample $\epsilon_i \sim \mathcal{N}(0, 1)$

$$z_i = h_i(\epsilon, \phi, x) = \sigma_i(x)\epsilon_i + \mu_i(x)$$

$$z \sim \mathcal{N}(\mu_{\phi}(x), \Sigma_{\phi}(x))$$

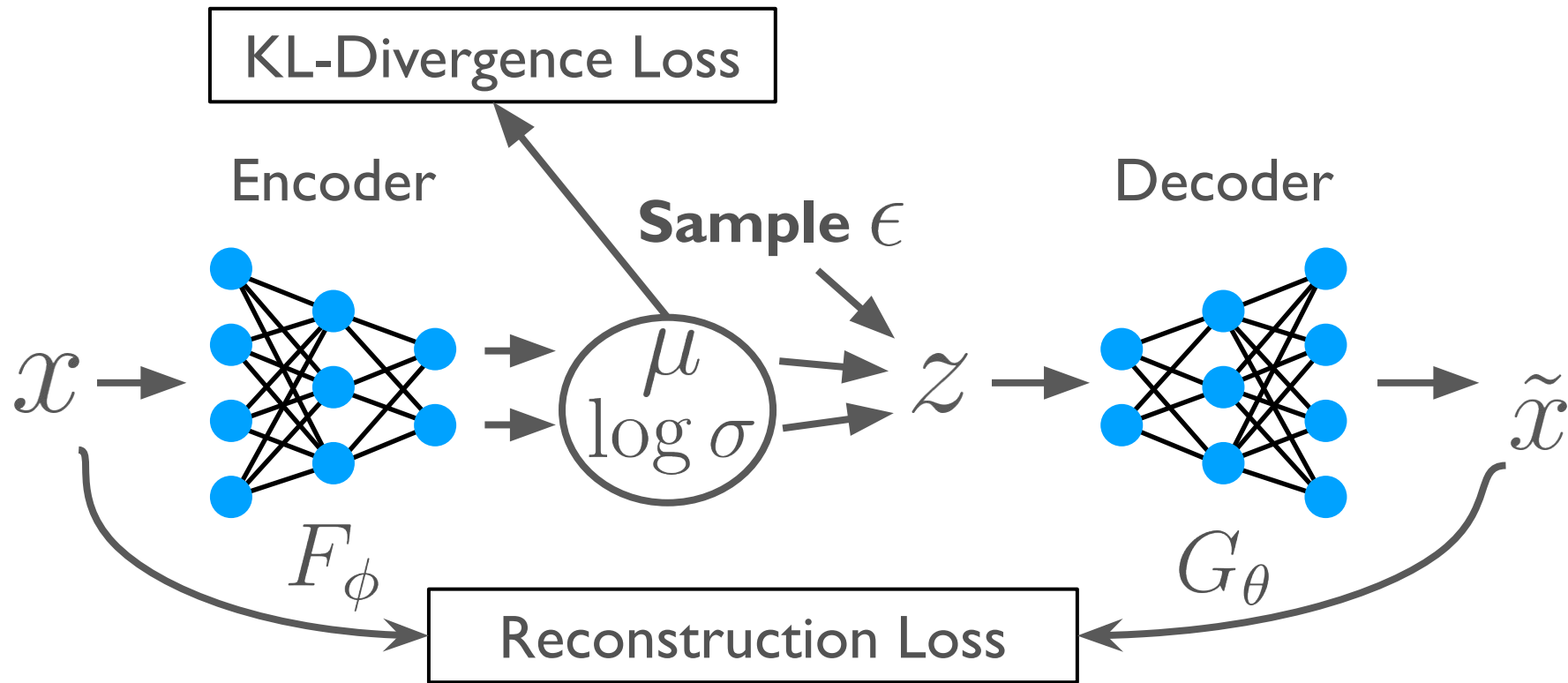
The reparameterization trick

$$\begin{aligned}\nabla_{\phi} \mathbb{E}_{q_{\phi}(z|x)} \|x - G_{\theta}(z)\|_2^2 &= \nabla_{\phi} \mathbb{E}_{p(\epsilon)} \|x - G_{\theta}(h(\epsilon, \phi, x))\|_2^2 \\ &= \mathbb{E}_{p(\epsilon)} \nabla_{\phi} \|x - G_{\theta}(h(\epsilon, \phi, x))\|_2^2\end{aligned}$$

Monte Carlo estimate (unbiased): single sample of ϵ

$$\nabla_{\phi} \|x - G_{\theta}(h(\epsilon, \phi, x))\|_2^2$$

Putting everything together



Here's what we'll need:

Architecture



Loss function



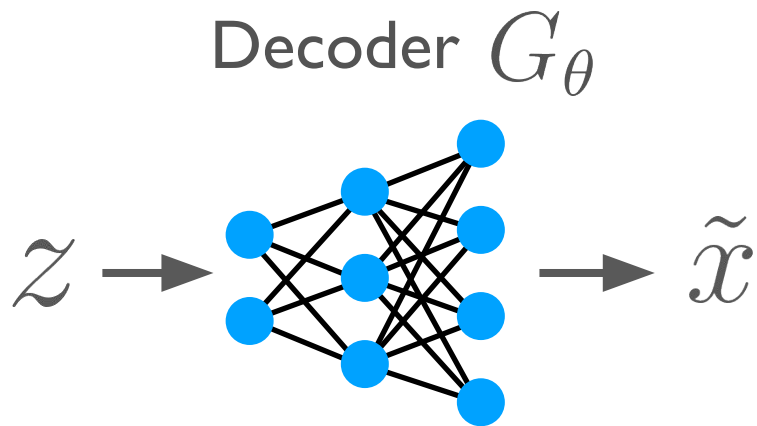
Gradients



Train with SGD

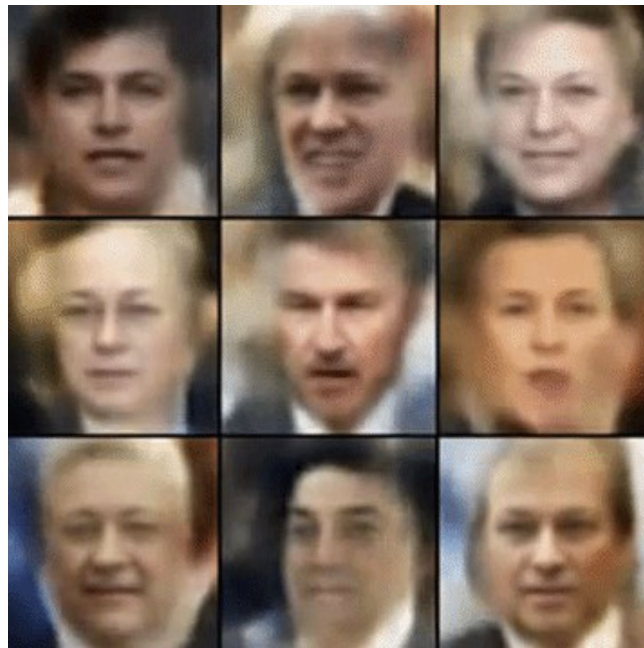
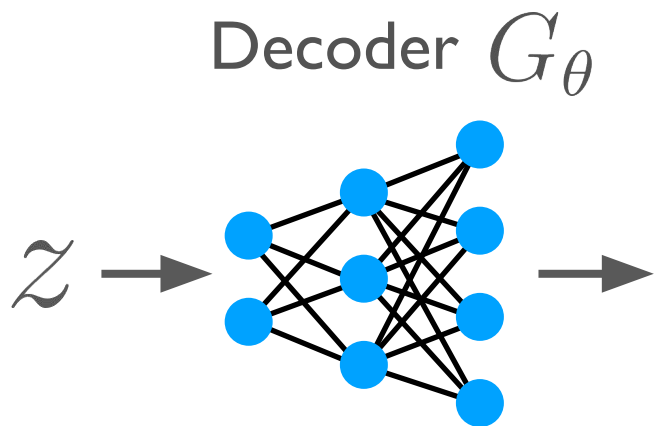


VAEs as generative models



Can use $z \sim \mathcal{N}(0, I)$ because of KL-divergence loss

VAEs as generative models



We will see that diffusion models are VAEs



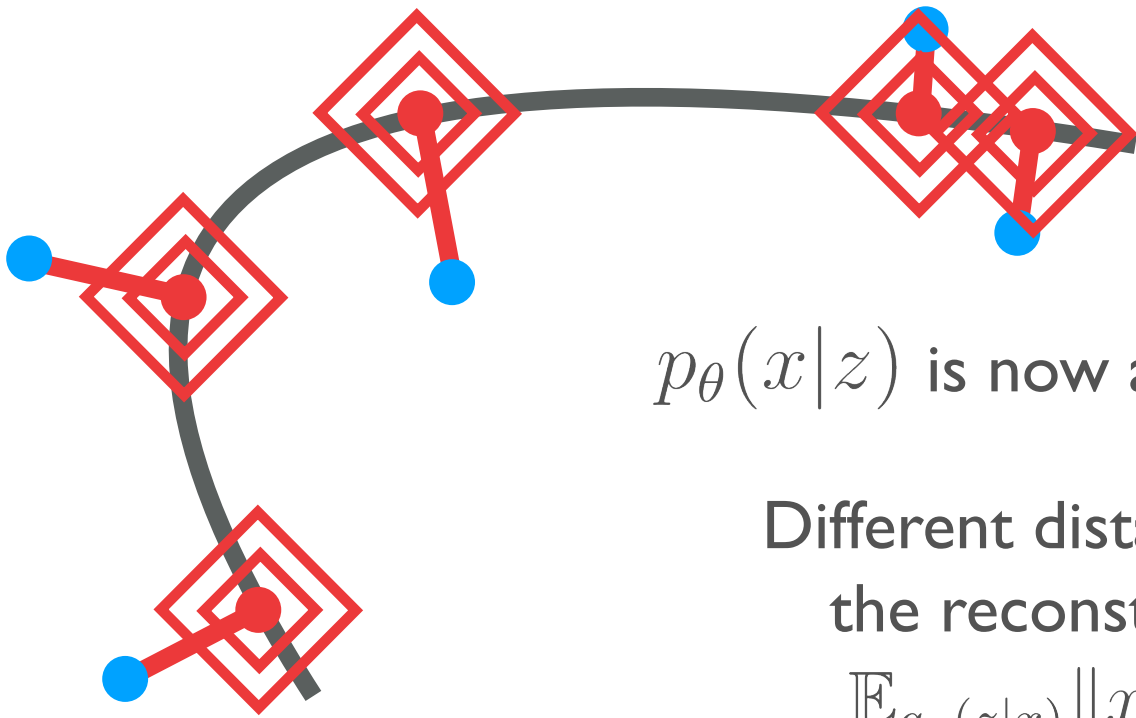
Non-Gaussian Observation Models

Define new $p_{\theta}(x|z)$

Just swap observation model in reconstruction loss

$$-\mathbb{E}_{q_{\phi}(z|x)} \log p_{\theta}(x|z)$$

Laplace distribution



$p_{\theta}(x|z)$ is now a Laplacian density

Different distance metric for
the reconstruction loss:

$$\mathbb{E}_{q_{\phi}(z|x)} \|x - G_{\theta}(z)\|_1$$

Discrete data

Example: binarized MNIST



Decoder outputs softmax probability $p_{\theta}(x_{ij}|z)$

Can view this as pixel-wise classification

$$p_{\theta}(x|z) = \prod_{i,j} p_{\theta}(x_{ij}|z)$$

Reconstruction loss $-\mathbb{E}_{q_{\phi}(z|x)} \log p_{\theta}(x|z)$ is just cross-entropy!

Non-Gaussian latent vectors

Can use other latent distributions

Reparameterization trick works for:

- Distributions with scale/shift parameters
- Tractable CDF

Discrete latent vectors

Map continuous inputs into discrete latent space

Map discrete latent into continuous output space

Differentiable?

What should loss function be?

Vector Quantized VAE (VQ-VAE) (van den Oord 2017)

- Store codebook of latent vectors (like LLM embedding vectors)
- Encoder outputs continuous feature vector
- Round feature vector to nearest latent vector from codebook
- Put latent vector into decoder and decode

Can have have encoder that outputs multiple feature vectors

E.g. 16x16 grid of vecs for image. Each vec gets latent

Vector Quantized VAE (VQ-VAE) (van den Oord 2017)

How to train? What loss function?

Three loss terms:

- Reconstruction loss
- Loss that pushes encoder output towards closest codebook vector
- Loss that pushes closest codebook vector towards encoder output

Don't need KL loss - uniform prior over codebook $\rightarrow$ constant KL loss

$$L = \log p(x|z_q(x)) + \| \text{sg}[z_e(x)] - e \|_2^2 + \beta \| z_e(x) - \text{sg}[e] \|_2^2$$

Vector Quantized VAE (VQ-VAE) (van den Oord 2017)

How to compute gradients of reconstruction loss?

Not differentiable because of quantization

Solution: straight-through estimator

Pretend the quantization layer is the identity in the backward pass

Accurate gradient whenever encoder output is closed to nearest latent vector

We have a loss term that ensures this!

Vector Quantized VAE (VQ-VAE) (van den Oord 2017)

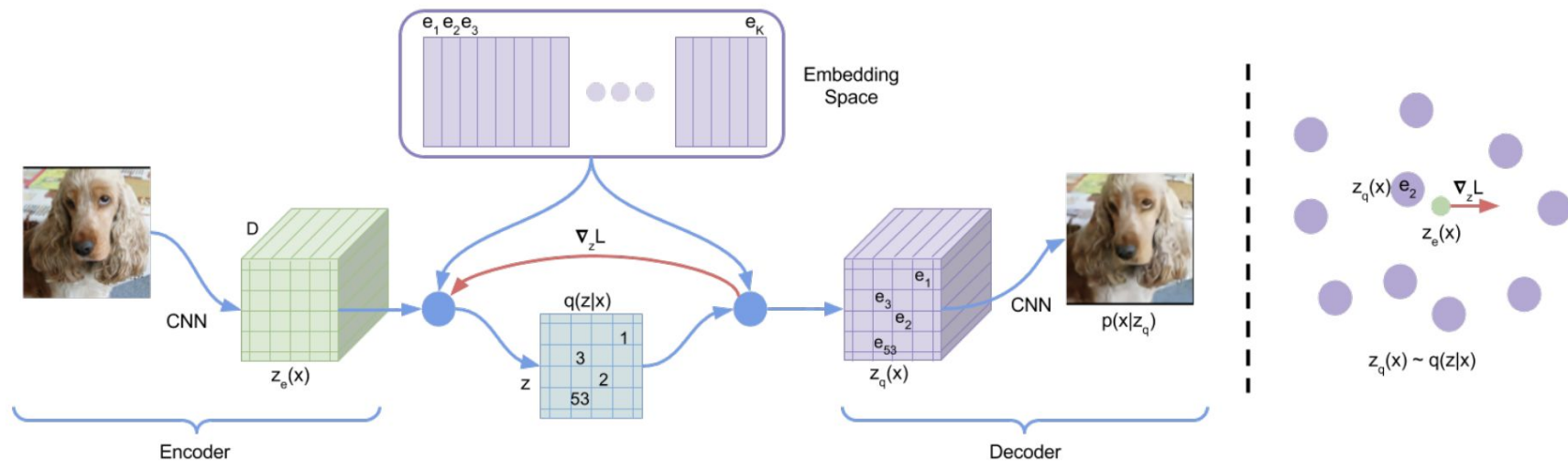


Figure 1: Left: A figure describing the VQ-VAE. Right: Visualisation of the embedding space. The output of the encoder $z(x)$ is mapped to the nearest point e_2 . The gradient $\nabla_z L$ (in red) will push the encoder to change its output, which could alter the configuration in the next forward pass.

Generative Adversarial Networks (Goodfellow 2014)

Problem: MSE loss $\rightarrow$ blurry images

If multiple outputs correspond to same latent, MSE loss $\rightarrow$ model outputs average

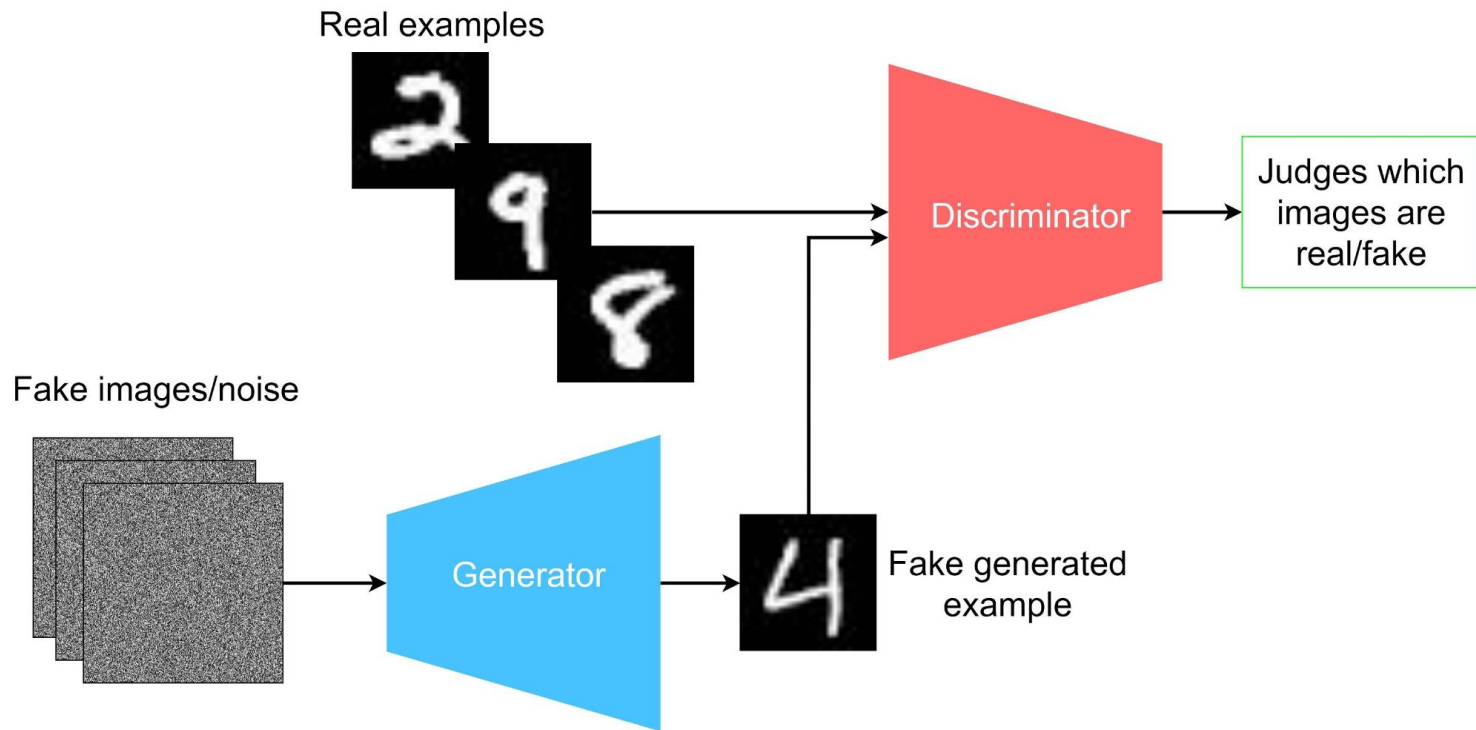
New type of generative model that avoids MSE loss: GAN

Two player game:

- Generator: take in noise vector, outputs image (or other kind of data)
- Discriminator: takes in image, predicts if real or generated

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

Generative Adversarial Networks (Goodfellow 2014)



Generative Adversarial Networks (Goodfellow 2014)

Two major areas of work:

Conditional generation:

- Class conditioning
- Text conditioning

Unstable saddle point optimization:

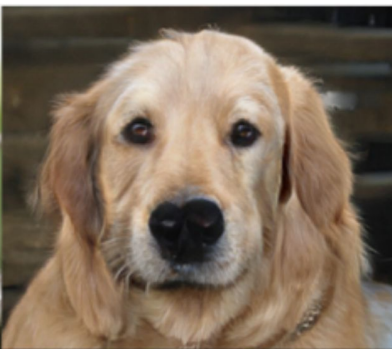
- Generator and discriminator need to train at same speed
- More stable loss functions

Scaled up architectures

StyleGAN-XL (Sauer 2022)



Jacamar



Golden Retriever



Boathouse



Photocopier



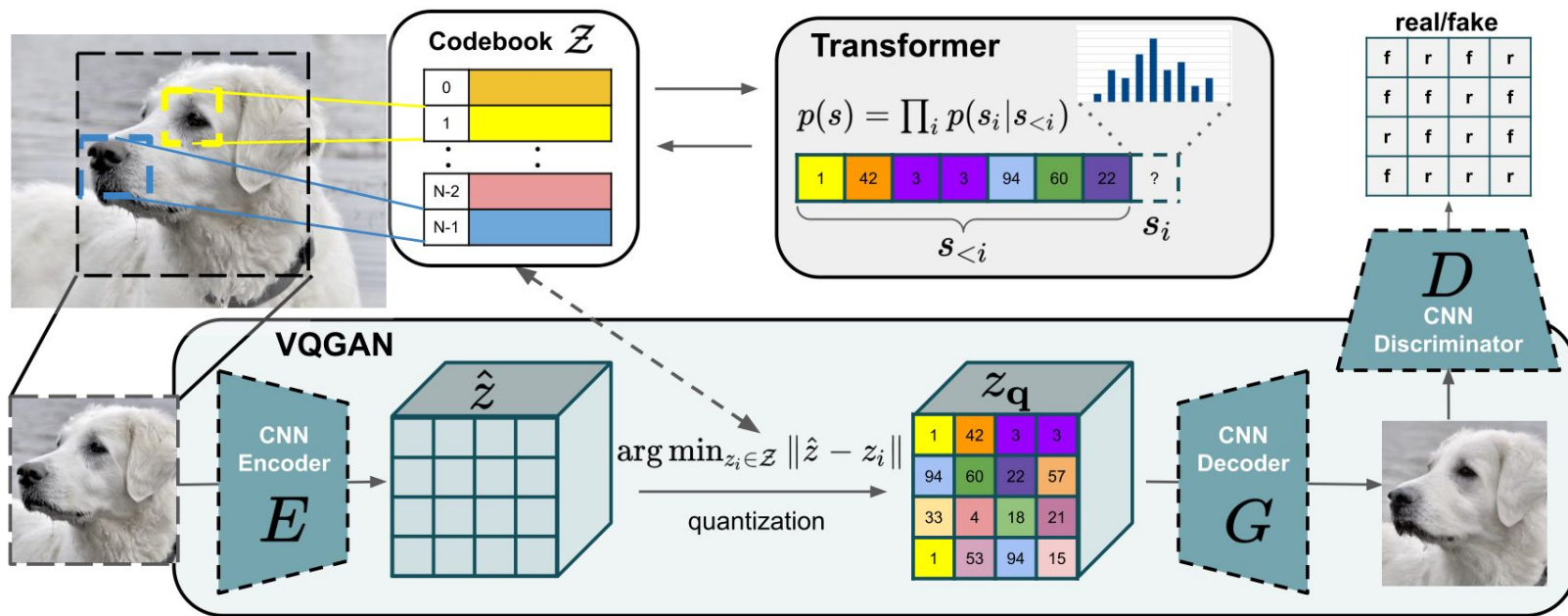
Trifle



Agaric

VQGAN (Esser 2021)

Can use discriminator to sharpen images generated by VQ-VAE



Autoregressive Generative Models for Images (Sun 2024)

VQ-VAE or VQ-GAN can map images in and out of discrete space

Treat discrete tokens like text tokens

Training:

- Run image dataset through encoder, get sequence of tokens
- Train LLM-style transformer with causal language modeling objective

Generation:

- Generate sequence of tokens using transformer
- Use VQ-GAN decoder to transform into image

Autoregressive Generative Models for Images (Sun 2024)



Figure 1: **Image generation with vanilla autoregressive models.** We show samples from our class-conditional image (top row) and text-conditional image (bottom row) generation models.

Multimodal modeling with VQGAN tokens

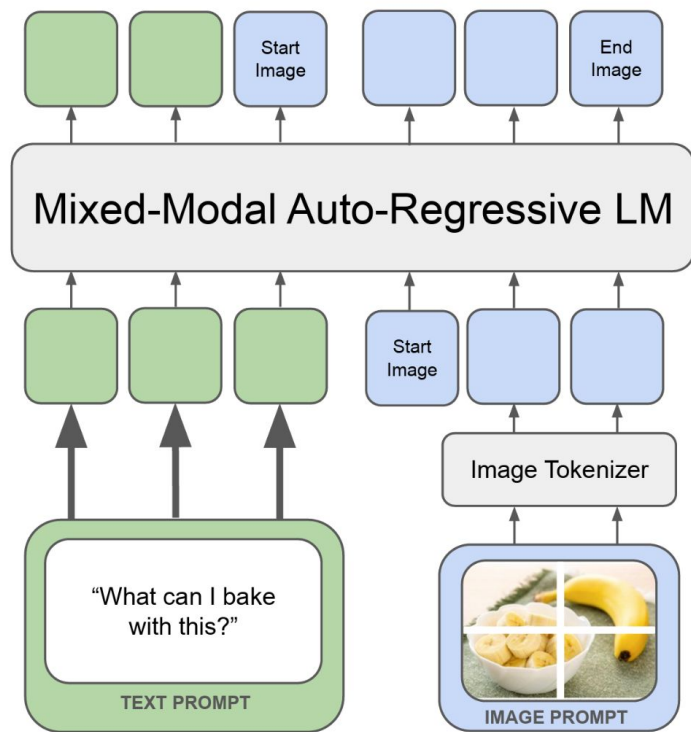
Previously:

- LLM vocab consists of text only
- Vision-language models (VLMs) used continuous vectors from an image feature extractor to take in images
- VLMs took in text/image input but only output text

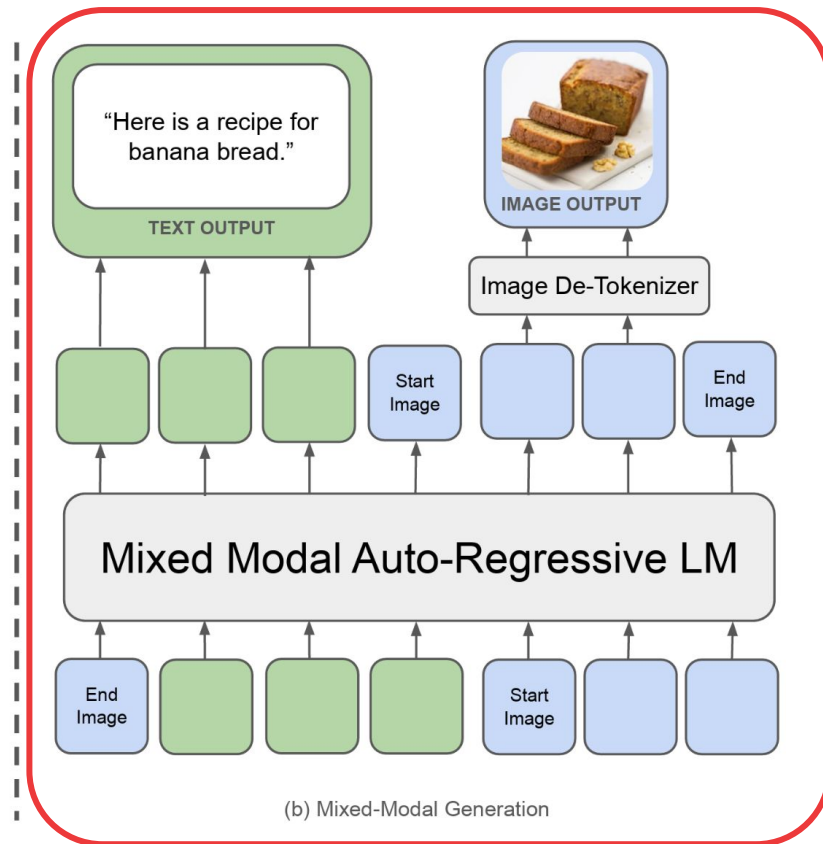
New idea:

- Add VQGAN tokens to LLM vocab
- Fine-tune model on interleaved text-image dataset
- Model outputs `start image` and `end image` tokens to switch
- Use VQGAN decoder to render images after generation

Chameleon (FAIR 2024)



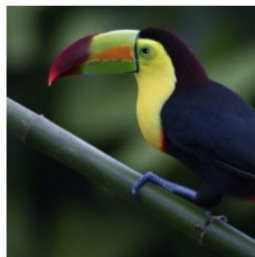
(a) Mixed-Modal Pre-Training



(b) Mixed-Modal Generation

Chameleon (FAIR 2024)

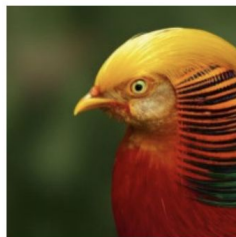
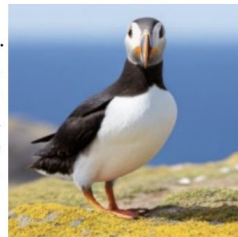
Prompt: I'm bored. Could you show me some cool, quirky-looking birds? Also tell me about them in 2-3 sentences.



Response: Absolutely! Here are three birds with some unique features that you might find interesting:

Keel-Billed Toucan The Keel-Billed Toucan is a brightly colored bird native to South American rainforests and woodlands. Its most distinctive feature is its colorful beak, which can grow up to seven inches long and has a distinctive, curved shape. The beak is used for plucking fruit from trees and for social displays. Despite its large size, the Keel-Billed Toucan is a skilled flyer and can often be seen flitting through the treetops. Here is an image of this vibrant bird: ****

Puffin Puffins are small, aquatic birds that are found in the northern hemisphere. They have distinctive black and white feathers, as well as a brightly colored beak that is used for attracting mates and defending their territory. Puffins are known for their distinctive "puffin" call, which sounds like a loud, guttural chuckle. They feed on fish, which they catch by diving into the water from the air. Puffins are colonial birds, and they breed in large colonies on rocky cliffs and islands. Here is an image of this adorable bird: ****



Golden Pheasant The Golden Pheasant is a brightly colored bird native to the forests of China. It is a member of the pheasant family and is known for its striking plumage, which features a golden-yellow body, red face, and green tail feathers. The Golden Pheasant is a ground-dwelling bird, and it feeds on a diet of insects, seeds, and fruits. It is a popular game bird, and is often hunted for its meat and feathers. In captivity, the Golden Pheasant is sometimes kept as an ornamental bird, due to its striking appearance. Here is an image of this beautiful bird: ****

I hope these images and descriptions have helped to brighten your day! If you have any other questions or requests, please don't hesitate to ask.

Figure 2 Sample interleaved image and text generation from Chameleon. The corresponding images are generated in locations marked by ****.